

Atelier 6 : l'outil complet en modules

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Chapitre** : 6, Organiser un programme · **Version** : 2026-10-23

Avant de commencer

Ce que vous allez construire	Le paquet `yonnee`, qui réunit le travail des ateliers 2 à 5 : un module par rôle (modèle, lecture, filtres, rapports, outils), un programme principal `main.py`, un environnement virtuel avec `requirements.txt`, et un journal à l'écran et dans un fichier
Durée	🟢 Débutant : 4 h · 🔵 Standard : 3 h · 🟣 Expert : 2 h (sans les défis)
Prérequis	Avoir lu le chapitre 6 ; votre dépôt `yonnee-express` (ateliers 2 à 5)
Fichiers fournis	Aucun nouveau : le registre d'octobre (atelier 2) et le registre complet (atelier 3), dans votre dossier `donnees`
Rendu	Votre dépôt GitHub `yonnee-express`, avec l'étiquette `atelier-6`

Votre dépôt contient quatre programmes qui se recopient les uns les autres. La direction veut **un seul outil**, facile à faire évoluer : si un tarif change, on le modifie à un seul endroit. Vous allez **réorganiser** le code existant, sans réécrire ce qui marche, et le **prouver** : à la fin, l'outil doit redonner exactement les chiffres des ateliers 2 et 4.

> 📍 **Point de départ** : votre dépôt de l'atelier 5. Vous allez **reprendre** des fonctions de `registre.py` (atelier 2), `rapports.py` (atelier 4) et `grand_registre.py` (atelier 5). Ne supprimez pas ces fichiers : ils restent dans l'historique et servent de référence.

Voici l'organisation visée :

```
yonnee-express/
├── .gitignore
├── requirements.txt
├── main.py              le programme principal
├── donnees/             les registres CSV (et tarifs.json)
└── yonnee/              le paquet
    ├── __init__.py      VERSION
    ├── modele.py        Livraison, tarifs, frais_livraison
    ├── lecture.py       charger_tarifs, convertir, lire_registre
    ├── filtres.py       les fabriques de conditions, filtrer
    ├── rapports.py      moyenne, rapport, en_tableau
    └── outils.py        chronometre
```

Module	Importe…	N'importe jamais…
`modele.py`	rien de `yonnee`	les autres modules du paquet
`lecture.py`	`modele`	`filtres`, `rapports`
`filtres.py`, `outils.py`	rien de `yonnee`	les autres modules
`rapports.py`	`tabulate` (paquet externe)	les autres modules
`main.py`	tout le paquet	(personne n'importe `main.py`)

Dans le paquet, seul `lecture.py` importe un autre module (`modele.py`), et `modele.py` n'importe rien du paquet : aucune boucle d'importation possible (chapitre 6, section 4.3). `main.py` est tout en haut : il importe tout, et personne ne l'importe.

Comment travailler

Mêmes règles que les ateliers précédents. **Règle de cet atelier** : chaque module, une fois écrit, doit pouvoir être **importé sans rien afficher**. Testez-le à chaque étape avec `python -c "import yonnee.nom_du_module"` (l'option `-c` lance le code donné entre guillemets) : aucune ligne ne doit s'afficher.

Étape 1 : l'environnement virtuel (20 min)

🎯 **Objectif** : un environnement propre au projet, et un dépôt qui n'embarque pas ce qui ne doit pas y être.

🛠️ **A faire**

1. Dans le dossier du dépôt : `python -m venv .venv`, puis activez-le (chapitre 6, section 6.2). Dans VS Code, choisissez l'interpréteur `.venv` (« Python: Select Interpreter »).
2. `pip install tabulate`, puis `pip freeze > requirements.txt`.
3. Complétez le `.gitignore` (créé à l'atelier 5) :

```
.venv/
__pycache__/
*.log
donnees/registre_grand.csv
sorties/
```

✅ **Vérification** : `(.venv)` au début de la ligne du terminal ; `requirements.txt` contient une ligne `tabulate==...` ; `git status` ne propose **pas** le dossier `.venv`.

💡 **A comprendre** : pourquoi met-on `requirements.txt` sur GitHub, mais pas `.venv` ? Qu'est-ce qu'un camarade doit taper pour faire tourner votre projet après l'avoir cloné ?

```
git add .gitignore requirements.txt
git commit -m "Environnement virtuel : requirements.txt et .gitignore"
```

Étape 2 : le modèle (20 min)

🎯 **Objectif** : un module qui contient les **données de base**, et rien d'autre.

🛠️ A faire

1. Créez le dossier `yonnee` et son `__init__.py`, avec une docstring et `VERSION = "1.0"`.
2. `yonnee/modele.py` : le tuple nommé `Livraison` (atelier 4), les constantes de l'atelier 2 (`TARIFS_PAR_DEFAUT`, `SUPPLEMENT_LOURD`, `POIDS_LOURD`, `MONTANT_MIN`, `MONTANT_MAX`) et la fonction `frais_livraison(tarifs, quartier, poids)`.
3. Une docstring en haut du module, et une pour la fonction.

✅ Vérification

```
python -c "import yonnee.modele"
python -c "from yonnee.modele import frais_livraison, TARIFS_PAR_DEFAUT;
        print(frais_livraison(TARIFS_PAR_DEFAUT, 'Pikine', 7))"
```

La première commande n'affiche rien ; la seconde affiche `3000`.

💡 **A comprendre** : `COLONNES` (la liste des noms de colonnes de l'atelier 2) n'est plus nécessaire. Quel attribut du tuple nommé donne la même liste (atelier 5, étape 6) ?

```
git add yonnee
git commit -m "Paquet yonnee : le modèle"
```

Étape 3 : la lecture, avec un journal (45 min)

🎯 **Objectif** : lire le registre une livraison à la fois, et **journaliser** les rejets au lieu de les afficher.

🛠️ **A faire** : `yonnee/lecture.py`.

1. Les importations : `csv`, `json`, `logging`, `date`, puis, avec une importation **relative**, ce dont vous avez besoin dans `modele`.
2. Sous les importations : `journal = logging.getLogger(__name__)`.
3. `charger_tarifs(chemin)` (atelier 2), mais les deux `print` deviennent des `journal.warning(...)`.
4. `convertir(ligne, tarifs)` (atelier 2), qui renvoie maintenant une `Livraison` au lieu d'un dictionnaire.
5. `lire_registre(chemin, tarifs, rejets=None)` : une **fonction génératrice** (chapitre 5) qui donne les livraisons valides une par une. Pour chaque ligne invalide : un `journal.warning("ligne %d rejetée : %s", numero_ligne, e)`, et, si `rejets` n'est pas `None`, l'ajout de `(numero_ligne, str(e))` à la liste. Au début de la lecture, un message `INFO` avec le nom du fichier.

▼ 💡 Indice : pourquoi `rejets=None` et pas `rejets=[]` ?

Relisez le chapitre 4, section 2.2 : une liste comme valeur par défaut serait partagée entre tous les appels. Avec `None`, celui qui appelle la fonction décide s'il veut récupérer les rejets, en passant **sa** liste.

✅ **Vérification**

```
python -c "import yonnee.lecture"
python -c "from yonnee.lecture import lire_registre; from yonnee.modele import TARIFS_PAR_DEFAUT
        as T; r = []; print(len(list(lire_registre('donnees/registre_octobre.csv', T, r))),
        r[:2])"
```

```
ligne 6 rejetée : could not convert string to float: '2,5'
ligne 13 rejetée : quartier inconnu : Ouakam
...
53 [(6, "could not convert string to float: '2,5'"), (13, 'quartier inconnu : Ouakam')]
```

Les 7 rejets s'affichent, puis 53 livraisons valides. Le message `INFO` du début de lecture n'apparaît pas : aucun réglage n'a encore été fait, et Python se contente alors d'afficher les messages `WARNING` et plus graves, **tels quels**, sans heure ni niveau. C'est le programme principal qui choisira la forme des messages (étape 6).

💡 **A comprendre**

1. Le module `lecture` n'appelle **jamais** `logging.basicConfig`. Pourquoi ? Qui doit le faire ?
2. A l'étape 6, chaque message commencera par `yonnee.lecture`. D'où viendra ce nom ?

```
git add yonnee
git commit -m "Lecture du registre avec journal des rejets"
```

Étape 4 : les filtres et les outils (20 min)

🛠️ **A faire**

1. `yonnee/filtres.py` : les cinq fabriques de l'atelier 4 et `filtrer`, qui renvoie maintenant une **expression génératrice** (chapitre 5) au lieu d'une liste.
2. `yonnee/outils.py` : le décorateur `chronometre` de l'atelier 5, mais la durée est écrite dans le journal du module au niveau `DEBUG` (`journal.debug("%s : %.3f s", fonction.__name__, duree)`), au lieu d'un `print` .

✅ **Vérification** : les deux modules s'importent sans rien afficher.

💡 **A comprendre** : `filtrer` renvoie maintenant un générateur. Que faut-il écrire pour connaître le **nombre** de livraisons filtrées ? Que se passe-t-il si on parcourt deux fois le même résultat de `filtrer` ?

```
git add yonnee
git commit -m "Filtres paresseux et chronomètre journalisé"
```

Étape 5 : les rapports en tableau (30 min)

🎯 **Objectif** : séparer le **calcul** d'un rapport de son **affichage**.

À l'atelier 4, `rapport` calculait **et** affichait. Une fonction qui fait deux choses est difficile à réutiliser (impossible d'avoir le résultat sans l'affichage). On les sépare.

🛠️ **A faire** : `yonnee/rapports.py` .

1. `moyenne(valeurs)` (atelier 4).
2. `rapport(livraisons, cle, valeur=lambda l: l.frais, agregat=sum)` : renvoie un dictionnaire groupe -> résultat, **trié** du plus grand au plus petit, sans rien afficher. Indice : `dict(sorted(resultat.items(), key=..., reverse=True))` .
3. `en_tableau(resultat, entetes, decimales=0)` : renvoie le texte du tableau, fabriqué par `tabulate` avec `tablefmt="github"` et `floatfmt=f".{decimales}f"` (le nombre de décimales des nombres à virgule).

✅ **Vérification** : dans le terminal Python (environnement actif) :

```
>>> from yonnee.rapports import en_tableau
>>> print(en_tableau({"Pikine": 1076000, "Yoff": 967000}, ["quartier", "frais (F)"]))
| quartier   |   frais (F) |
|------------|-------------|
| Pikine     |     1076000 |
| Yoff       |      967000 |
```

💡 **A comprendre** : `rapports.py` est le seul module qui dépend d'un paquet **externe**. Qu'affiche `import yonnee.rapports` si l'environnement virtuel n'est pas actif ? Et `import yonnee.lecture` ? Pourquoi est-ce une bonne chose que seul ce module dépende de `tabulate` ?

```
git add yonnee
git commit -m "Rapports : calcul et affichage séparés"
```

Étape 6 : le programme principal (30 min)

🎯 **Objectif** : un `main.py` qui règle le journal et enchaîne les traitements.

🛠️ **A faire** : `main.py`, à côté du dossier `yonnee`.

1. `configurer_journal()` : deux destinations (chapitre 6, section 7.3) ; l'écran à partir de `INFO`, le fichier `journal.log` (recréé à chaque exécution, en UTF-8) pour **tout**, `DEBUG` compris. Format : `"%(asctime)s %(levelname)-8s %(name)s : %(message)s"`, heure seule.
2. Une fonction `charger(chemin, tarifs, rejets)`, décorée par `@chronometre`, qui renvoie `list(lire_registre(chemin, tarifs, rejets))`.
3. `main()` : régler le journal ; afficher la version ; charger les tarifs (`donnees/tarifs.json`) ; lire le registre d'octobre, puis le registre complet, en affichant chaque fois le nombre de livraisons valides et rejetées ; afficher les tableaux des frais par quartier et de la durée moyenne par livreur (une décimale) ; les retards à Pikine ; les colis de plus de 8 kg livrés par Awa en 2026.
4. `if __name__ == "__main__": main()`.

✅ **Vérification** : `python main.py`.

```
Yonnee Express, outil version 1.0
Octobre : 53 livraisons valides, 7 rejetées
Registre complet : 2000 livraisons, 0 rejetées

Frais par quartier
| quartier   |   frais (F) |
|------------|-------------|
| Pikine     |     1076000 |
| Yoff       |      967000 |
| Parcelles  |      894000 |
| Plateau    |      729000 |
| Médina     |      506000 |

Durée moyenne par livreur
| livreur   |   minutes |
|-----------|-----------|
| Ibou      |      45.6 |
| Awa       |      45.1 |
| Fatou     |      45.0 |
| Modou     |      44.5 |
| Coumba    |      44.0 |

Retards à Pikine : 109
Colis de plus de 8 kg livrés par Awa en 2026 : 54
```

Les messages du journal (en `INFO` et `WARNING` : les lectures et les 7 rejets) s'affichent aussi ; selon le terminal, ils peuvent apparaître avant ou entre ces lignes, car le journal écrit sur une autre « sortie » que `print` (la sortie des erreurs). Si vous n'avez pas de `tarifs.json`, un `WARNING` le signale et les tarifs par défaut sont utilisés.

💡 **A comprendre** : comparez avec les ateliers 2 et 4 : quels chiffres retrouvez-vous ? Pourquoi est-ce la meilleure preuve que la réorganisation n'a rien cassé ?

```
git add main.py
git commit -m "Programme principal et réglage du journal"
```

Étape 7 : le journal et la livraison (15 min)

🛠️ A faire

1. Ouvrez `journal.log` : combien de lignes ? Retrouvez les lignes `DEBUG` du chronomètre : combien de temps a pris la lecture de chaque registre ?
2. Relancez `python main.py` : le fichier a-t-il grossi ? Pourquoi ?
3. Clonez votre dépôt dans un autre dossier (`git clone ...`), refaites-y l'environnement (`python -m venv .venv` , activer, `pip install -r requirements.txt`), copiez-y les deux registres et lancez `python main.py` . Tout doit marcher.

✅ **Vérification** : `journal.log` a 12 lignes (le tarif par défaut, deux lectures, 7 rejets, deux durées) et n'en a toujours que 12 après une deuxième exécution ; le projet cloné tourne.

```
git add .
git commit -m "Outil Yonnee Express en paquet"
git tag atelier-6
git push
git push origin atelier-6
```

🏆 Défis (facultatifs)

1. **`python -m yonnee`.** Un fichier `__main__.py` dans un paquet est exécuté par `python -m nom_du_paquet` . Créez `yonnee/__main__.py` , qui lit un registre et affiche les frais par livreur. Quelle forme doivent prendre ses importations (relatives ou non) ?
2. **Des options.** Avec le module `argparse` (bibliothèque standard), faites accepter à `python -m yonnee` le chemin du registre, une option `--quartier` (ne garder qu'un quartier) et une option `--verbeux` (afficher les messages `INFO`). Que fait `python -m yonnee -h` ? et `python -m yonnee` sans registre ? Combien de retards avec `donnees/registre_complet.csv --quartier Pikine` ?
3. **Le grand registre.** Ajoutez à `main.py` une option pour traiter le grand registre de l'atelier 5 **sans** liste (directement le générateur `lire_registre`) : le total des frais doit valoir 1 048 561 000 F.
4. 🟣 **Le README.** Écrivez un `README.md` qui explique à un nouveau développeur comment installer et lancer l'outil, et qui décrit le rôle de chaque module.

Grille d'autoévaluation

Je sais…	Oui	Pas encore
découper un programme en modules qui s'importent sans rien afficher	☐	☐
organiser les importations sans boucle	☐	☐
créer un paquet et utiliser des importations relatives	☐	☐
créer un environnement virtuel et le reproduire avec `requirements.txt`	☐	☐
journaliser avec des niveaux, à l'écran et dans un fichier	☐	☐
vérifier qu'une réorganisation ne change aucun résultat	☐	☐